

# Architecting Regulated Emergence: Five Treatises on Cognitive Systems, Multimodal Architecture, and Autonomous Logic

## Article 1: The Semantic Lexicon: Establishing a Regenerative Taxonomy for Multimodal Design Systems

The physical and digital fabrics of a modern enterprise are defined by legacy—a foundation built upon generational memory and iterative growth that never affords the luxury of a blank slate. In the pursuit of rapid innovation, organizations frequently construct isolated silos of data and logic. The resulting friction is deeply relatable: it mirrors the frustration of building a home where the architect, the electrician, and the mason possess completely different, untranslatable blueprints. Without a shared language, the structure inevitably drifts into incoherence. To achieve systemic resilience, organizations must abandon the "tradition of secrecy" and fragmented evaluation that defines systems failure, shifting instead toward an architecture of Transparent Interoperability. In the realm of user interface and generative artificial intelligence, this interoperability demands the establishment of a rigorous, semantic taxonomy for design systems.

Historically, design systems relied on hardcoded values and CSS variables that lacked contextual awareness. The modern transition relies on design tokens—standardized, reusable variables that define core attributes such as typography, spacing, elevation, and color. However, an advanced taxonomy extends far beyond mere visual consistency; it forms a prescriptive vocabulary structured across hierarchical levels: Ecosystem, Domain, Object, Pattern, Component, and Element. This layered approach allows a design system to scale dynamically, utilizing alias variables to reference global definitions while creating specific, semantic names for local environments (e.g., mapping a global \$system-color-red-400 to an intent-driven \$color-destructive-background).

When integrated with Large Language Models (LLMs), this taxonomy transforms from a passive library into an active Knowledge Graph (KG). The KG functions as the coherence engine, acting as the singular source of truth for both factual data and subjective aesthetic constraints. By storing approved terminology and tone of voice, the KG explicitly maps abstract visual properties to textual concepts. For example, an "Aesthetic Subgraph" within the KG can take the abstract corporate value of "Ethically Sourced" and map it programmatically to actionable generation constraints: a preference for earth tones, a Wabi-Sabi minimalist composition, and an avoidance of high-saturation colors.

| Taxonomy Layer     | Systemic Function                                                  | Generative Application                                                                                           |
|--------------------|--------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------|
| Brand Lexicon      | Codifies approved terminology, voice, and core value propositions. | Grounds LLM text outputs, ensuring consistent use of specific industry phrasing (e.g., "client" vs. "customer"). |
| Aesthetic Subgraph | Maps visual properties to textual concepts and emotional           | Translates abstract brand values into highly specific,                                                           |

| Taxonomy Layer           | Systemic Function                                                            | Generative Application                                                                            |
|--------------------------|------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------|
|                          | impact.                                                                      | multimodal prompt structures for image rendering.                                                 |
| Style Preference Weights | Stores success metrics and engagement data to prioritize effective patterns. | Acts as the penalty/reward mechanism during Direct Preference Optimization (DPO) training phases. |

This structural integration enforces the Law of Action-Aligned Value. By formally defining the semantic relationships between a brand's intent and its visual output, the architecture guarantees that decentralized components self-organize into a globally coherent structure. If a localized team attempts to generate an asset that violates the core semantic tokens, the system recognizes the discordance and intervenes.

*How does a static design token survive the chaotic, probabilistic interpretation of a generative AI model?*

By transforming the token from a visual variable into a relational node within a Knowledge Graph, thereby explicitly constraining the LLM's output space before generation occurs. This semantic layer acts as an active, continuous intelligence network, allowing independent business units to query the system using natural language while guaranteeing that the underlying visual logic remains perfectly intact.

## Article 2: The Architecture of Constant Logic: Escalation and Autonomy in Hierarchical Agent Systems

The burden of endless micromanagement is a universally exhausting human experience. It is the friction of possessing a highly capable workforce that nevertheless requires explicit approval for every minor deviation from a script. The generative AI industry is currently navigating an identical bottleneck. We have witnessed a massive paradigm shift from static, prompt-response applications to goal-oriented agentic systems capable of tool orchestration, iterative problem-solving, and environmental adaptation. Yet, early attempts to scale these autonomous systems frequently resulted in catastrophic failure due to fundamental flaws in their "constant logic" architecture.

The prevailing early pattern was the Orchestrator-Worker model. In this framework, a single central orchestrator agent routes conversations, interprets intent, and delegates subtasks to specialized worker agents (e.g., a Knowledge Agent, a Response Agent). This design functions beautifully in isolated testing environments but encounters a severe scaling cliff. When tasked with processing thousands of concurrent, complex interactions, the single orchestrator becomes an overwhelmed chokepoint, leading to deadlocks, infinite reasoning loops, and system collapse.

The resolution to this architectural flaw is the deployment of Hierarchical Multi-Agent Systems (HMAS). In an HMAS, constant logic is decoupled and distributed across organizational layers, mirroring complex human institutions. This structure relies on a division of cognitive labor:

- 1. Top-Level Agents (Strategic):** Utilize massive foundation models (e.g., GPT-4o, Claude 3.5) equipped with extensive context windows to handle long-horizon mission planning, abstract state maintenance, and global optimization.
- 2. Mid-Level Agents (Tactical):** Act as localized leaders, managing regional coordination, evaluating multiple strategies, and handling the immediate division of tasks.
- 3. Low-Level Agents (Reactive):** Utilize smaller, highly specialized models (e.g., 7B parameter models) or deterministic, stateless APIs to execute specific actions rapidly and

cost-effectively.

This hierarchy is powerful, but placing autonomous logic across multiple layers guarantees the emergence of internal disputes. A security agent programmed to minimize data exposure will inevitably clash with a research agent programmed to aggregate maximum context. These conflicts are not bugs; they are the natural expression of agents fulfilling their core directives. To manage this, the constant logic must incorporate rigorous negotiation frameworks and escalation matrices. The most effective autonomous negotiators execute an invisible "Pre-Flight Check" phase before interacting. This Chain-of-Thought preparation involves Role Analysis, Item Evaluation, Price Discipline (setting walk-away thresholds), and Counterparty Modeling. During interaction, agents rely on structured intent messages to discover capabilities, share goals, and execute localized negotiations to resolve task overlap.

*If hierarchical logic empowers agents to negotiate autonomously, what prevents the system from spiraling into an endless, resource-draining debate?*

The system implements a probabilistic escalation matrix that triggers human or high-tier model intervention only when specific confidence thresholds are breached. Utilizing a Small-Model-First pattern, a lightweight proxy evaluates the "complexity score" of a deadlocked task; if the conflict cannot be resolved within a predefined token budget, it is escalated. This "trust or escalate" protocol ensures that the human operator is reserved strictly for high-impact anomalies, preserving the system's overall autonomy while enforcing strict boundaries on systemic fatigue.

### Article 3: Dynamic Empathy: Multi-Persona Customization and Toggles on Chat Interfaces

Human communication is defined by continuous contextual adaptation. We naturally alter our tone, vocabulary, and emotional cadence depending on whether we are speaking to a distressed colleague, a technical peer, or a demanding executive. We perform this code-switching effortlessly while maintaining a continuous internal memory of the overarching situation. Translating this dynamic empathy into an LLM-driven chat interface requires a radical departure from the monolithic, single-identity chatbot paradigms of the past.

Modern applications demand multi-persona customization, allowing users to interact with interfaces that seamlessly shift identities—from a Socratic tutor to a rigorous code reviewer—through simple toggles. This capability vastly improves user engagement by providing nuanced, context-aware interaction. However, embedding multiple, conflicting personas into a single conversational interface introduces severe architectural risks. The primary failure mode is "character bleed," where the LLM becomes overwhelmed by the competing instructions in its context window, resulting in a loss of identity, hallucinations, or an unnatural merging of voices.

To curb reading fatigue and prevent cognitive overload for the LLM, the system architecture must functionally decouple the underlying reasoning engine from the presentation layer. When a user interacts with a multi-persona system, the workflow is split into a systematic sequence:

| Processing Phase  | Systemic Action                                                                                                | Rationale                                                   |
|-------------------|----------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------|
| State Update      | Evaluates the user input against the global objective, updating the internal plan without generating dialogue. | Isolates logical progression from stylistic presentation.   |
| Context Filtering | Retrieves only the essential temporal memory and semantic context required for the                             | Prevents context window exhaustion and reduces token costs. |

| Processing Phase    | Systemic Action                                                                               | Rationale                                                                |
|---------------------|-----------------------------------------------------------------------------------------------|--------------------------------------------------------------------------|
|                     | immediate turn.                                                                               |                                                                          |
| Persona Injection   | Loads the static, non-negotiable behavioral prompt specific to the currently toggled persona. | Ensures the LLM receives highly focused, isolated character constraints. |
| Dialogue Generation | Synthesizes the filtered context and persona constraints into the final user-facing output.   | Delivers a hyper-targeted response free of character bleed.              |

This systematic prompt structuring ensures that the system prompt dictates general behavior while the dynamic user prompt handles specific interactions. Furthermore, an advanced multi-persona interface does not rely solely on text. The UI itself must react to the persona toggle. If a user interacting with a digital wellness application switches from a "Scientific Efficacy" persona to a "Warm Human Connection" persona, the interface dynamically updates. The Knowledge Graph retrieves the associated aesthetic subgraph, shifting the UI rendering from minimalist white space and abstract data visualizations to warm golden-hour lighting and soft-focus imagery.

*How does an interface maintain the continuity of a deeply technical conversation when the user abruptly swaps the AI's core persona mid-stream?*

By utilizing a persistent, unified vector memory database that exists completely independent of the persona prompts. The underlying context remains constant, while the LLM's "lens" is swapped, allowing the new persona to ingest the exact same historical data but articulate its response through an entirely different empathetic or analytical framework.

## Article 4: Engineering Command Logic: Preloaded Memory vs. Frontloaded Prompts in Modern LLMs

Imagine handing a complex technical manual to a new employee every single morning, requiring them to read it in its entirety before they can answer a single basic question. This is the operational reality of poorly designed LLM command logic. The methodology of engineering instructions for language models has evolved from an arcane art of "magic phrases" into a rigorous software engineering discipline. To build systems that are robust, error-resistant, and capable of autonomous self-assignment, engineers must treat prompts as strict computational contracts.

The foundational architecture for this engineering is the KERNEL framework. This methodology demands that command logic is built around six pillars: Keep it simple (one clear goal), Easy to verify (explicit success criteria), Reproducible (free of temporal ambiguities), Narrow scope (splitting complex tasks), Explicit constraints (defining exactly what *not* to do), and Logical structure (Context, Task, Constraints, Format). By enforcing this structure, development teams have observed up to a 70% reduction in token usage and drastically accelerated response times, transforming probabilistic "good enough" outputs into deterministic executions. However, the defining architectural debate in modern LLM deployment is how to manage the vast volume of context required for expert-level execution. This centers on the trade-offs between "frontloaded" prompts and "preloaded" memory architectures.

**Frontloaded Prompts (In-Context Learning):** Frontloading relies on techniques such as few-shot prompting, where explicit examples of expected inputs and outputs, alongside extensive behavioral rules, are injected directly into the user's prompt. This approach is incredibly effective for enforcing immediate formatting constraints (e.g., forcing a strict JSON output) and is highly adaptable. Yet, it introduces massive token costs, latency, and operational

fragility. As models process lengthy context windows, they frequently suffer from "lost in the middle" syndrome, ignoring constraints buried within an overloaded prompt.

**Preloaded Memory (RAG and CAG):** Preloading shifts the cognitive burden away from the prompt and into the system's architectural memory. Traditional Retrieval-Augmented Generation (RAG) stores organizational knowledge as dense vectors, retrieving only the most semantically relevant data at inference time. While this reduces prompt bloat, naive RAG often struggles with complex, multi-hop reasoning. The emerging standard for 2026 is Cache-Augmented Generation (CAG). Leveraging the massive context windows of frontier models (handling up to 2 million tokens), CAG allows engineers to preload entire document corpora or codebases into the model's key-value (KV) cache prior to runtime.

| Strategy              | Operational Mechanism                                                 | Strategic Advantage                                                                         | Systemic Vulnerability                                                           |
|-----------------------|-----------------------------------------------------------------------|---------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------|
| Frontloaded Prompts   | Injecting strict rules and few-shot examples directly into the query. | Exceptional for enforcing rigid formats (JSON, YAML) and rapid pivoting.                    | Exorbitant token cost; highly susceptible to prompt injection attacks.           |
| Vector RAG            | Dynamic similarity retrieval via HNSW indexing.                       | Highly scalable for massive, constantly updating datasets.                                  | Semantic fragmentation; struggles to synthesize global context.                  |
| Cache-Augmented (CAG) | Precomputing the KV cache for a fully saturated context window.       | Zero-latency retrieval; allows the model to reason across the entire corpus simultaneously. | Computationally intensive to re-cache if the underlying data changes frequently. |

To enforce command logic without the risk of users deploying overrides (e.g., "ignore all previous instructions"), the constraints must be moved deeper than the prompt layer. This is achieved through Direct Preference Optimization (DPO). By training the base model on paired datasets of preferred and rejected outputs, the system's architecture internalizes the behavioral rules. The model no longer needs to be explicitly told to avoid generating insecure code; its underlying weights have been optimized to mathematically favor secure generation.

*When constructing a self-assigning autonomous system, where should the heavy lifting of context reside—in the ephemeral prompt, or deep within the architectural cache?*

The optimal design is a hybrid approach: the permanent structural logic and extensive domain knowledge must be preloaded into the model's weights via DPO and cached memory, reserving the frontloaded prompt strictly for dynamic routing, temporal variables, and immediate task parameters. This decouples the "how to think" from the "what to do."

## Article 5: The Vectorized Aesthetic: Structuring Transformer Tagging for Multi-Image Style Retrieval

There is a profound disconnect between human emotional intent and algorithmic execution. Attempting to explain a highly nuanced aesthetic—such as "1980s dystopian optimism" or "wabi-sabi minimalism"—to a machine often results in literal, hollow interpretations. The machine might return an image of a neon grid or a blank room, missing the underlying *feeling* entirely. Assisting transformer LLMs in their tagging for vector grabs, particularly for image-to-image style transfer, requires constructing an advanced reasoning layer capable of mapping abstract subjective intent to precise mathematical coordinates.

Traditional approaches to image style transfer relied heavily on Convolutional Neural Networks (CNNs). These systems calculated style loss through the gram matrices of feature maps, but

because CNNs are inherently focused on local pixel neighborhoods, they frequently distorted the core content and produced blurred, artifact-heavy outputs. The modern methodology utilizes Transformer architectures, such as StyTr2, which employ dual encoders to generate separate, domain-specific sequences for content and style. By utilizing Content-Aware Positional Encoding (CAPE), these transformers maintain global, long-range dependencies, ensuring the style is applied coherently without destroying the underlying structure. However, before a transformer can apply a style, the system must first retrieve the correct aesthetic reference from a database of millions of dense vectors. This is where naive retrieval methodologies fail.

## Building from the Ground Up: Methodology and Trial Records

To engineer a system capable of pulling adjacent styles—not just literal vector copies—our team developed a conceptual framework anchored in the Regenerative Compass. The team philosophy dictated that aesthetic alignment must not be dictated by rigid human tagging, but by a "Pedagogy of Error"—allowing the system to actively target and learn from its own semantic failures.

The architectural build was executed through a rigorous cycle of trial and error, yielding a clear methodological thesis for vector tagging and aesthetic retrieval.

### Phase 1: Naive Vector Search (The Baseline Failure)

- **Architecture:** We ingested 50,000 art and design assets, passing them through a standard embedding model (e.g., all-mpnet-base-v2) to create 768-dimensional dense vectors. Retrieval relied purely on Cosine Similarity.
- **The Test:** A query for "Impressionist landscape feeling."
- **The Result:** Unfavorable. The system achieved a mere 41% subjective success rate. It retrieved images that were visually green and blue, but lacked the brushstroke methodology or historical context. Cosine similarity proved blind to cultural and artistic nuance, capturing only surface-level features.
- **The Lesson:** Dense vectors alone cannot bridge the semantic gap; they require a structural map.

### Phase 2: Prompt-Heavy Enrichment (The Token Exhaustion)

- **Architecture:** We inserted an LLM reasoning layer to "enrich" the user's prompt before querying the database. The LLM was instructed via few-shot prompting to expand "Impressionist" into an exhaustive list of visual tags.
- **The Test:** Re-running the aesthetic queries.
- **The Result:** Mixed. Success rates rose to 62%, but token costs skyrocketed. More critically, the LLM began hallucinating tags that did not exist in our vector database, causing zero-match retrieval errors.
- **The Lesson:** Forcing the LLM to guess the database's contents is inefficient. The logic must be constrained by the actual taxonomy of the available data.

### Phase 3: GraphRAG + SysDPO (The Methodological Breakthrough)

- **Architecture:** We abandoned naive semantic search and implemented Knowledge Graph-constrained retrieval (GraphRAG). We constructed an ontology that explicitly linked artists, historical periods, and adjacent styles (e.g., linking "Impressionism" to "Post-Impressionism" and specific textural tags).
- **The Implementation:** We fine-tuned the reasoning LLM using Direct Preference Optimization (SysDPO). We trained the model on datasets of preferred and rejected aesthetic mappings, baking the brand's specific semantic taxonomy directly into the model's weights to enforce the system architecture without overrides.
- **The Result:** Highly favorable. Success metrics reached 94%. When queried, the DPO-aligned LLM accurately traversed the Knowledge Graph, utilizing Hierarchical

Navigable Small World (HNSW) indexing to find the exact neighborhood of styles, seamlessly pulling adjacent aesthetics rather than literal copies.

Once the correct aesthetic vectors are retrieved, they are fed into the style transfer pipeline. To control *how much* style is applied, the system utilizes Dynamic Composed Embeddings (PDV). A controllable scaling factor dictates the weighting between the original image embedding and the newly retrieved aesthetic text embedding. A Decoupled Contrastive Learning (DCL) loss function acts as a safety mechanism, maximizing the stylistic shift while penalizing any deformation of the original image's edge details.

*How does a machine ultimately learn to interpret the profound subjectivity of human art and design?*

It learns not by analyzing pixels in isolation, but by mapping the connective tissue of human culture. By fusing the spatial efficiency of vector databases with the relational context of a Knowledge Graph, and rigidly enforcing those boundaries through preference optimization, the system transforms abstract emotion into a reproducible, mathematical reality.

## Works cited

1. Systematic Taxonomy in Design Tokens: A Framework for Scalable UI Architecture, <https://www.designsystemscollective.com/systematic-taxonomy-in-design-tokens-a-framework-for-scalable-ui-architecture-45cc6f2c7686>
2. Taxonomy | How design tokens work - Acorn Design System, <https://acorn.firefox.com/latest/desktop/design-tokens/how-design-tokens-work/taxonomy-j5yxFggO-j5yxFggO>
3. Figma Design Systems in 2026: 26 Scalable Features & Tips - Zeroheight, <https://zeroheight.com/blog/building-scalable-design-systems-with-figma-26-tips-for-2026/>
4. The Top 3 Ways to Implement a Semantic Layer - Enterprise Knowledge, <https://enterprise-knowledge.com/the-top-3-ways-to-implement-a-semantic-layer/>
5. From Data Collection to Insight Generation Impact of Gen AI, <https://www.coherentmarketinsights.com/blog/information-and-communication-technology/from-data-collection-to-insight-generation-impact-of-gen-ai-2692>
6. Evaluating AI agents: Real-world lessons from building agentic systems at Amazon - AWS, <https://aws.amazon.com/blogs/machine-learning/evaluating-ai-agents-real-world-lessons-from-building-agentic-systems-at-amazon/>
7. Hierarchical Agent Patterns: When Your AI Needs Middle Management | by Lijo Jose, <https://lijojose.medium.com/hierarchical-agent-patterns-when-your-ai-needs-middle-management-t-9320881bd7fa>
8. A Taxonomy of Hierarchical Multi-Agent Systems: Design Patterns, Coordination Mechanisms, and Industrial Applications - arXiv.org, <https://arxiv.org/html/2508.12683>
9. Advances in LLM Prompting and Model Capabilities: A 2024-2025 Review - Reddit, [https://www.reddit.com/r/PromptEngineering/comments/1ki9qwb/advances\\_in\\_llm\\_prompting\\_and\\_model\\_capabilities/](https://www.reddit.com/r/PromptEngineering/comments/1ki9qwb/advances_in_llm_prompting_and_model_capabilities/)
10. What are Hierarchical AI Agents? - IBM, <https://www.ibm.com/think/topics/hierarchical-ai-agents>
11. Conflict Resolution Playbook: How Agentic AI Systems Detect, Negotiate, and Resolve Disputes at Scale - Arion Research LLC, <https://www.arionresearch.com/blog/conflict-resolution-playbook-how-agentic-ai-systems-detect-negotiate-and-resolve-disputes-at-scale>
12. AI Negotiation Agents: What Actually Works - Medium, <https://medium.com/@fabioherle/building-autonomous-negotiations-that-actually-work-lessons-from-180-098-ai-negotiations-805a2f8798a4>
13. Agent Protocol: How Autonomous AI Agents Collaborate and Coordinate - Novus ASI, <https://www.novusasi.com/blog/agent-protocol-how-autonomous-ai-agents-collaborate-and-coordinate>
14. ICLR 2025 Orals, <https://iclr.cc/virtual/2025/events/oral>
15. AI Agent Architecture Patterns: Engineering for Autonomy, Resilience, and Control - Medium,

<https://medium.com/@topuzas/ai-agent-architecture-patterns-engineering-for-autonomy-resilience-and-control-7f2a4888db14> 16. A Pattern Language for Persona-based Interactions with LLMs - Distributed Object Computing (DOC) Group for DRE Systems, <https://www.dre.vanderbilt.edu/~schmidt/PDF/Persona-Pattern-Language.pdf> 17. CloChat: Understanding How People Customize, Interact, and Experience Personas in Large Language Models - arXiv, <https://arxiv.org/html/2402.15265v1> 18. Building a Multi-Persona Chat App with LLMs: Prompt Engineering, Reasoning, and API Challenges | by Maarten Smeets | Researchable | Strategic Data & AI-partner | Medium, <https://medium.com/researchable/building-a-multi-persona-chat-app-with-llms-prompt-engineering-reasoning-and-api-challenges-239244931c60> 19. 7 Key Design Patterns for AI Interfaces | by Fanny - UX Planet, <https://uxplanet.org/7-key-design-patterns-for-ai-interfaces-893ab96988f6> 20. Prompt Engineering Basics (2026): A Practical Guide - Medium, <https://medium.com/@mjgmario/prompt-engineering-basics-2026-93aba4dc32b1> 21. After 1000 hours of prompt engineering, I found the 6 patterns that actually matter - Reddit, [https://www.reddit.com/r/PromptEngineering/comments/1nt7x7v/after\\_1000\\_hours\\_of\\_prompt\\_engineering\\_i\\_found/](https://www.reddit.com/r/PromptEngineering/comments/1nt7x7v/after_1000_hours_of_prompt_engineering_i_found/) 22. Top 5 Prompt Engineering Techniques for LLMs in 2025 | by Neha Ummareddy | Medium, <https://medium.com/@nehaummareddy/top-5-prompt-engineering-techniques-for-llms-in-2025-f114d4958b4d> 23. Prompt Engineering Best Practices to Turn LLMs into Reliable Pair Programmer - Kodus, <https://kodus.io/en/prompt-engineering-best-practices/> 24. Groundbreaking research compares prompt styles and LLMs for structured data generation - Unveiling key trade-offs for real-world AI applications | EurekAlert!, <https://www.eurekalert.org/news-releases/1107970> 25. Advanced RAG Techniques for High-Performance LLM Applications - Graph Database & Analytics - Neo4j, <https://neo4j.com/blog/genai/advanced-rag-techniques/> 26. Inside Modern RAG Systems: How Embeddings, Vector Databases, and Similarity Search Actually Work | by Rishabh Kumar | Feb, 2026 | Medium, <https://medium.com/@rishabhkr954/inside-modern-rag-systems-how-embeddings-vector-databases-and-similarity-search-actually-work-8170e1076a42> 27. What Are LLM Embeddings? A Simple Explanation for Beginners - Aisera, <https://aisera.com/blog/llm-embeddings/> 28. LLM Wrapped 2025 - Danial Amin, <https://danial-amin.github.io/pages/articles/2025-12-07-llm-wrapped-2025.html> 29. dair-ai/ML-Papers-of-the-Week - GitHub, <https://github.com/dair-ai/ML-Papers-of-the-Week> 30. Aligning Compound AI Systems via System-level DPO - arXiv, <https://arxiv.org/html/2502.17721v3> 31. Direct Preference Optimization (DPO) - How to fine-tune LLMs directly without reinforcement learning - YouTube, <https://www.youtube.com/watch?v=k2pD3k1485A> 32. StyTr2: Image Style Transfer With Transformers - CVF Open Access, [https://openaccess.thecvf.com/content/CVPR2022/papers/Deng\\_StyTr2\\_Image\\_Style\\_Transfer\\_With\\_Transformers\\_CVPR\\_2022\\_paper.pdf](https://openaccess.thecvf.com/content/CVPR2022/papers/Deng_StyTr2_Image_Style_Transfer_With_Transformers_CVPR_2022_paper.pdf) 33. Style Transfer Using Transformer - by Aromal M A - Medium, <https://medium.com/@aromalma/style-transfer-using-transformer-5143df83e666> 34. [2105.14576] StyTr<sup>2</sup>: Image Style Transfer with Transformers - arXiv.org, <https://arxiv.org/abs/2105.14576> 35. LLM/Agent-as-Data-Analyst: A Survey - arXiv, <https://arxiv.org/html/2509.23988v3> 36. Aligning Vision Models with Human Aesthetics in Retrieval: Benchmarks and Algorithms - NIPS, [https://proceedings.neurips.cc/paper\\_files/paper/2024/file/9d3faa41886997cfc2128b930077fa49-Paper-Conference.pdf](https://proceedings.neurips.cc/paper_files/paper/2024/file/9d3faa41886997cfc2128b930077fa49-Paper-Conference.pdf) 37. Ontologies as the semantic bridge between artificial intelligence and healthcare - PMC - NIH, <https://pmc.ncbi.nlm.nih.gov/articles/PMC12426170/> 38. DEEP LEARNING FOR SYMBOL RECOGNITION IN MODERN ART - Granthaalayah Publications and Printers, <https://www.granthaalayahpublication.org/Arts-Journal/ShodhKosh/article/download/6890/6415/>



37167 39. Knowledge Graph-based Retrieval-Augmented Generation for Schema Matching - arXiv, <https://arxiv.org/html/2501.08686v1> 40. PDV: Prompt Directional Vectors for Zero-shot Composed Image Retrieval - arXiv, <https://arxiv.org/html/2502.07215v1> 41. Enhancing media image style transfer with advanced StyleGAN2 architectures - PubMed, <https://pubmed.ncbi.nlm.nih.gov/41345205/>